

BİLGİSAYAR MÜHENDİSLİĞİ YÜKSEK LİSANS

BSM585 ÖNERİ SİSTEMLERİ

ÖDEV 1

Collaborative Filtering: User-Based ve Item-Based Yöntemlerin Karşılaştırmalı Analizi

Teslim Tarihi	19.04.2026
Format	Jupyter Notebook (.ipynb) + PDF Rapor
Programlama Dili	Python 3.x NumPy, Pandas, Matplotlib ✓ Surprise X
Çalışma Ortamı	Google Colab (tavsiye edilir) veya yerel Python ortamı
Bireysel / Grup	Bireysel

⚠ ÖNEMLİ UYARI: Kütüphane Yasağı

Surprise gibi hazır öneri sistemi kütüphaneleri kesinlikle kullanılamaz. PCC ve ACS benzerlik metrikleri NumPy / Python temel işlemleriyle sıfırdan yazılmalıdır. Bu kurala uymayan ödevler değerlendirmeye alınmayacaktır.

💻 Çalışma Ortamı Tavsiyesi: Google Colab

Leave-One-Out CV ile 4 farklı k değeri deneneceğinden toplam çalışma süresi yerel bir bilgisayarda 30–90 dakikaya ulaşabilir.

Google Colab ücretsiz GPU/CPU kaynakları sayesinde bu süreyi önemli ölçüde kısaltmaktadır.

Colab linki: <https://colab.research.google.com> — Google hesabınızla giriş yapıp yeni bir notebook açın.

Not: Yerel ortamda da çalıştırabilirsiniz; ancak çalışma sürelerini mutlaka raporda belirtin.

1. Amaç ve Genel Bakış

Bu ödev kapsamında MovieLens 100K veri seti üzerinde iki farklı Collaborative Filtering (CF) yöntemi sıfırdan geliştirilecektir: User-Based CF (PCC benzerliği) ve Item-Based CF (ACS benzerliği). Farklı komşu boyutları (k) test edilerek MAE metrikleri karşılaştırılacak; veri yoğunluğunun (sparsity) model performansına etkisi deneysel olarak incelenecektir.

Ödevin temel hedefleri:

- PCC ve ACS benzerlik metriklerini sıfırdan, doğru biçimde implemente etmek
- Leave-One-Out Cross-Validation uygulamak ve MAE hesaplamak
- $k = \{10, 25, 50, 100\}$ değerleri için karşılaştırmalı deney yürütmek ve çalışma sürelerini ölçmek
- %20 veri silme senaryosuyla sparsity etkisini analiz etmek
- Bulguları grafik, tablo ve akademik düzeyde yorumla raporlamak

2. Veri Seti

2.1 MovieLens 100K

Veri setini <https://grouplens.org/datasets/movielens/> adresinden indirin. 943 kullanıcı, 1682 film, yaklaşık 100.000 oy (1–5 tam sayı) içermektedir.

2.2 Zorunlu Veri Ön İşleme

Aşağıdaki adımları Notebook içinde Markdown açıklamalarıyla gerçekleştirin; her karar gerekçelendirilmelidir:

- Mükerrer (user_id, item_id) çiftlerinin tespiti ve nasıl ele alındığının açıklanması
- Eksik değer (NaN) sayısının raporlanması
- Sparsity oranının hesaplanması: $\text{sparsity} = 1 - (|R| / (|U| \times |I|))$
- Bu sparsity değerinin CF yöntemleri üzerindeki etkisinin yorumlanması (en az 3 cümle)

3. Kısım 1: User-Based Collaborative Filtering (PCC)

3.1 Benzerlik Fonksiyonu: Pearson Korelasyon Katsayısı (PCC)

PCC, aktif kullanıcı a ile diğer kullanıcılar arasındaki benzerliği ortak değerlendirilen ürünler üzerinden ölçer. Formülü slaytlardan elde edebilirsiniz.

Zorunlu Fonksiyon İmzası

Aşağıda verilen fonksiyon şablonunu kullanmanız zorunludur. Gövde (body) tamamen sizin tarafınızdan doldurulacaktır:

```
def compute_pcc(ratings_matrix, user_a, user_u):  
    """  
    Aktif kullanıcı user_a ile kullanıcı user_u arasındaki PCC değerini hesaplar.  
    Params:
```

```
ratings_matrix : pd.DataFrame — satırlar kullanıcı, sütunlar film,
                    değerler oy (NaN = oylanmamış)
user_a      : int — aktif kullanıcı ID'si
user_u      : int — karşılaştırılan kullanıcı ID'si
Returns:
float — PCC değeri [-1, 1]; ortak ürün sayısı < 2 ise 0.0
"""
# TODO: implement
pass
```

Ortak oy sayısı < 2 olan kullanıcı çiftleri için 0.0 döndürünüz.

3.2 Tahmin Fonksiyonu

User based prediction formülünü ilgili slayttan elde edebilirsiniz.

Zorunlu Fonksiyon İmzası

```
def predict_user_based(ratings_matrix, user_a, item_i, k, similarity_func):
    """
    User-based CF ile user_a kullanıcısının item_i ürününe vereceği oyu tahmin eder.
    Params:
        ratings_matrix : pd.DataFrame
        user_a      : int
        item_i      : int
        k          : int — komşu sayısı
        similarity_func: callable — compute_pcc veya başka benzerlik fonksiyonu
    Returns:
        float | None — tahmin edilen oy; üretilmiyorsa None
    """
    # TODO: implement
    pass
```

3.3 Değerlendirme: Leave-One-Out CV

Protokol

1. Her kullanıcı sırayla aktif kullanıcı olarak seçilir.
2. Aktif kullanıcının oylarından 5 film rastgele seçilir (random seed = 42).
3. Bu 5 filmten 1 tanesi test ürünü olarak ayrılır; gerçek oy saklanır, veriden geçici olarak çıkarılır.
4. Kalan 4 film ve diğer kullanıcıların verileri eğitim kümesini oluşturur.

5. Tahmin üretilemediğinde (yeterli komşu yoksa) kullanıcı MAE hesabına dahil edilmez; bu durum sayılır ve raporda belirtilir.

MAE

Tahmin edilen değerler (p_i) ile gerçek değer (o_i) arasındaki hata MAE ile hesaplanır. R ise tahmin edilen test verilerinin sayısıdır. Unutulmamalıdır ki, düşük MAE sonuçları daha iyi sonuçları gösterir.

$$MAE = \frac{1}{R} \sum_{i=1}^R |p_i - o_i|$$

Her kullanıcı için ayrı MAE hesaplayınız, ardından tüm kullanıcıları kapsayan global MAE değerini bulunuz.

3.4 k Değeri Deneyleri ve Süre Ölçümü

$k = \{10, 25, 50, 100\}$ değerleri için sistemi ayrı ayrı çalıştırınız. Her deney için hem MAE hem de toplam çalışma süresini (saniye) ölçünüz. Süre ölçümü için Python'ın time veya datetime modülünü kullanabilirsiniz.

Metrik	k = 10	k = 25	k = 50	k = 100
Global MAE	???	???	???	???
Toplam Süre (sn)	???	???	???	???
Çalışma Ortamı	Colab / Yerel — (hangisini kullandığınızı belirtin)			

- $k \rightarrow$ MAE değerlerini çizgi grafikte gösterin; en iyi ve en kötü noktaları işaretleyin.
- $k \rightarrow$ Süre değerlerini ayrı bir çizgi grafikte gösterin.
- Eğrinin neden bu şekli aldığını underfitting/overfitting analojisi ile yorumlayınız. (En az 150 kelime)

4. Kısım 2: Item-Based Collaborative Filtering (ACS)

4.1 Benzerlik Fonksiyonu: Adjusted Cosine Similarity (ACS)

ACS, kullanıcı ortalamasını çıkararak bireysel oy önyalığını gidererek filmler arası benzerliği ölçer. ACS denklemini ilgili slayttan elde edebilirsiniz.

Zorunlu Fonksiyon İmzası

```
def compute_acs(ratings_matrix, item_i, item_j):
    """
    item_i ile item_j filmleri arasındaki ACS değerini hesaplar.
    Params:
        ratings_matrix : pd.DataFrame — satırlar kullanıcı, sütunlar film
        item_i         : int — birinci film ID'si
        item_j         : int — ikinci film ID'si
    Returns:
        float — ACS değeri [-1, 1]; ortak kullanıcı < 2 ise 0.0
    """
    # TODO: implement
    pass
```

4.2 Tahmin Fonksiyonu

Item based predcition formülünü kullanınız.

Zorunlu Fonksiyon İmzası

```
def predict_item_based(ratings_matrix, user_u, item_i, k, similarity_matrix):
    """
    Item-based CF ile user_u kullanıcısının item_i ürününe vereceği oyu tahmin eder.
    Params:
        ratings_matrix : pd.DataFrame
        user_u         : int
        item_i         : int
        k              : int — en benzer film sayısı
        similarity_matrix: pd.DataFrame — önceden hesaplanmış ACS matrisi
    Returns:
        float | None — tahmin edilen oy; üretilmiyorsa None
    """
    # TODO: implement
    pass
```

İpucu: Item-item benzerlik matrisini bir kez hesaplayıp saklayın; her tahmin çağrısında yeniden hesaplamak çok yavaş olacaktır.

4.3 Deneyler, Süre Ölçümü ve Karşılaştırma

Aynı Leave-One-Out protokolünü (seed = 42) uygulayarak $k = \{10, 25, 50, 100\}$ değerleri için MAE ve çalışma sürelerini hesaplayınız. Sonuçları Kısım 3 ile aynı formatındaki bir tabloya kayıt ediniz.

- User-Based ve Item-Based MAE değerlerini tek grafik üzerinde, farklı renk/sembolle gösterin.
- User-Based ve Item-Based sürelerini de ayrı bir karşılaştırmalı grafikte sunun.
- Hangi yöntem hangi k değerinde daha başarılı ve daha hızlıdır? MovieLens veri setinin özelliklerine ($|U|$, $|I|$, sparsity) atıfla yorumlayınız. (En az 200 kelime)

5. Veri Silme Senaryosu — Sparsity Analizi

Bu bölümde veri eksikliğinin (sparsity artışı) model performansına etkisi deneysel olarak incelenir.

5.1 Deney Adımları

6. Orijinal ~100.000 oylu MovieLens 100K veri setini yükleyin.
7. Eğitim verisinden rastgele %20'lik bir kısmı silin (seed = 42). Test verisine dokunmayın.
8. Yeni sparsity oranını hesaplayın ve orijinaliyle karşılaştırın.
9. Kısım 3 ve 4'te belirlediğiniz en iyi k değeriyle her iki CF yöntemini yeniden çalıştırın.
10. MAE ve süre değerlerini orijinal deney sonuçlarıyla karşılaştırın.

5.2 Grafik ve Sorular

- Grouped bar chart: Orijinal ve %20 silinmiş veri için User-Based / Item-Based MAE değerlerini yan yana gösterin. Başlık, eksen etiketleri ve legend zorunludur.

Raporunuzda kendi çıktılarınıza dayanarak aşağıdaki soruları yanıtlayınız:

- %20 veri silme işlemi MAE'yi ne kadar değiştirdi? Beklediğinizden farklı mıydı?
- User-based ve item-based yöntemler veri eksikliğine farklı tepki verdi mi? Neden?

6. Analiz Soruları

Tüm yanıtlar kendi uygulamanızdan elde ettiğiniz sonuçlara dayanmalıdır. Genel teorik açıklamalar puan almaz.

11. En yüksek MAE değerine sahip 3 kullanıcıyı bulunuz. Oy sayılarını, ortalamalarını ve dağılımlarını inceleyerek sistem neden bu kullanıcılar için düşük performans gösterdiğini açıklayabilir misiniz?

12. `compute_pcc` ve `compute_acs` fonksiyonlarının zaman karmaşıklığını (time complexity) analiz ediniz. Ölüştüğünüz çalışma süreleri bu analizinizi doğruluyor mu?
13. Elde ettiğiniz MAE değerleri ve hesaplama maliyetleri üzerinden, gerçek bir üretim sisteminde User-Based mi yoksa Item-Based CF mi tercih edilmeli? Deneylerinize dayanarak savununuz.

7. Teknik Gereksinimler

7.1 Kod Kalitesi

- PCC ve ACS sıfırdan (NumPy/Python) yazılmalıdır. Hazır CF kütüphaneleri kesinlikle yasaktır.
- Tüm fonksiyonlarda docstring (""" ... """) zorunludur: ne yaptığı, parametreler, dönüş değeri.
- Tüm rastgele işlemler için `random seed = 42` kullanınız.
- Notebook baştan sona hatasız çalışmalıdır (Kernel → Restart & Run All). Çalışmayan notebook'lar değerlendirme dışı bırakılabilir.

7.2 Süre Ölçümü

Her k değeri ve her yöntem için çalışma süresini ölçünüz. Örnek kullanım:

```
import time
start = time.time()
# ... CF kodu buraya ...
elapsed = time.time() - start
print(f"Süre: {elapsed:.2f} saniye")
```

- Tüm süre ölçümleri tablolara ve grafiklere yansıtılmalıdır.
- Colab ve yerel ortam süreleri farklılık gösterebilir; kullandığınız ortamı raporda mutlaka belirtin.

7.3 Grafik Standartları

- Tüm grafikler başlık, eksen etiketleri ve legend içermelidir.

7.4 Rapor Formatı

- Jupyter Notebook (.ipynb): Tüm kodlar, grafikler ve Markdown açıklamalar. Her bölüm için Markdown başlıkları kullanınız.

- PDF Rapor: Analiz sorularına verilen cevaplar PDF’te yer almalıdır. Tablolar ve grafikler de dahil edilmelidir.

8. Teslim

Aşağıdaki dosyaları tek bir ZIP arşivi olarak sisteme yükleyiniz:

- OgrenciNo_Ad_Soyad_CF.ipynb — Jupyter Notebook
- OgrenciNo_Ad_Soyad_CF.pdf — PDF rapor

Notebook baştan sona çalıştırılabilir olmalıdır. Çalışmayan ya da eksik çıktı üreten notebook’lar değerlendirme dışı bırakılabilir.

Hazır CF kütüphanesi kullanan ödevler değerlendirmeye alınmayacaktır.

Her parçanın nasıl çalıştığını açıklayabilmeniz beklenmektedir.